

Data Mining (EECS 6412)

Graph Neural Networks

Instructor: Aijun An

EECS, York University

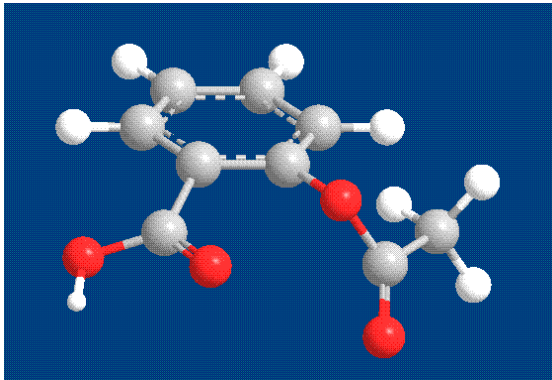
Outline of Introduction

- ▶ Why graph neural networks (GNNs)?
- ▶ Graph fundamentals
- ▶ Graph mining tasks
- ▶ Graph representation methods
- ▶ Graph embeddings
- ▶ GNN basics
- ▶ How to train a GNN
- ▶ Advantages of GNNs

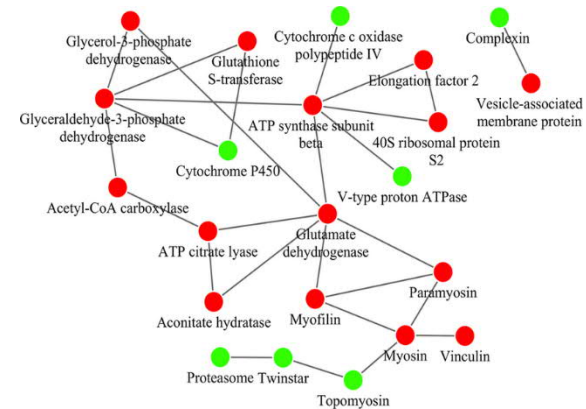
Graph Data

- ▶ Data entities are connected, forming graphs

Aspirin



Biological Networks

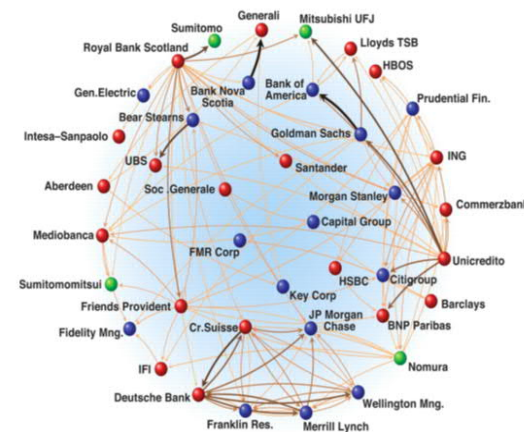


Credits: Chinese Academy of Agricultural Sciences

Social Networks



Financial Networks



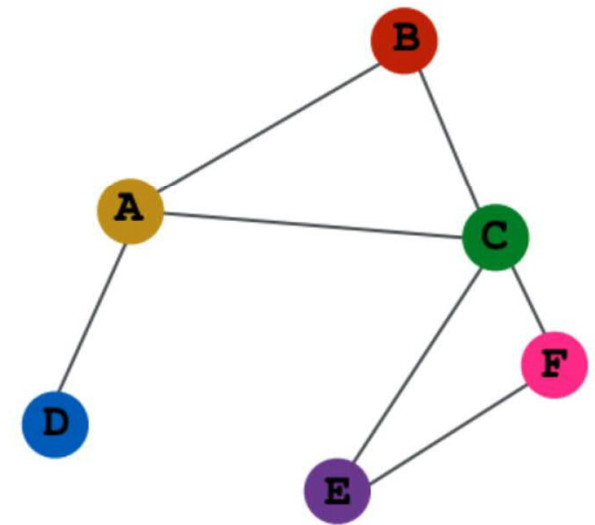
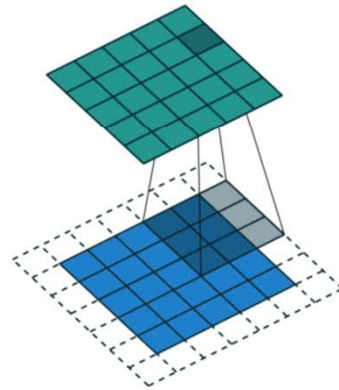
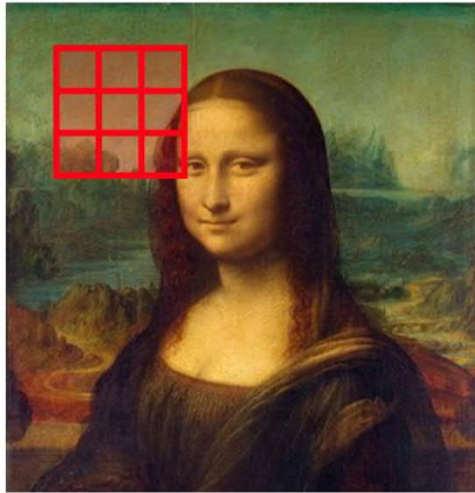
Sample of the international financial network
Credits: ETH Zurich

Motivation for Graph Neural Networks (GNNs)

- ▶ Neural networks do not usually consider the relationships between entities, except
 - ▶ CNNs assumes Euclidean structures (grids) of images
 - ▶ RNNs assume sequential relationships between elements of an input sequence
- ▶ Graphs are non-Euclidean and irregular
 - ▶ Need models that respect relationships.
- ▶ GNNs generalize deep learning to graphs via message passing.

Images vs Graphs

From image to graph



- Constant number of neighbors
- Fixed ordering of neighbors

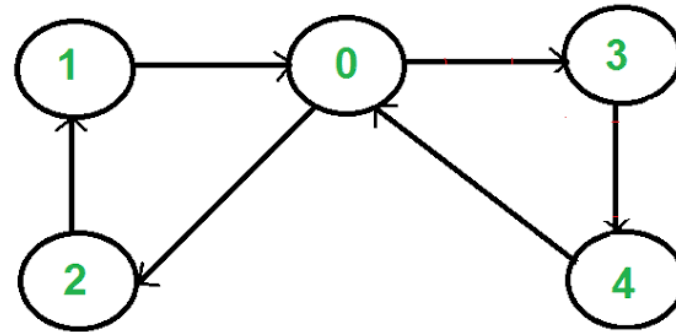
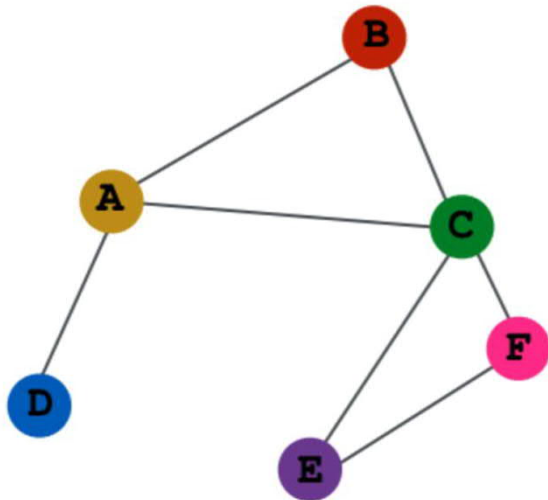
- Different number of neighbors
- No ordering of neighbors

Basic Concepts of Graphs

- ▶ A graph is defined as $G = (V, E)$
 - ▶ V is a set of nodes (representing entities)
 - ▶ E is a set of edges (representing relationships)
- ▶ Node can have attributes:
 - ▶ Features that describe the node (e.g., user age, paper topic)
- ▶ Edge can have attributes:
 - ▶ Features that describe the edge (e.g., weights, types, directions, timestamps).

Basic Concepts of Graphs

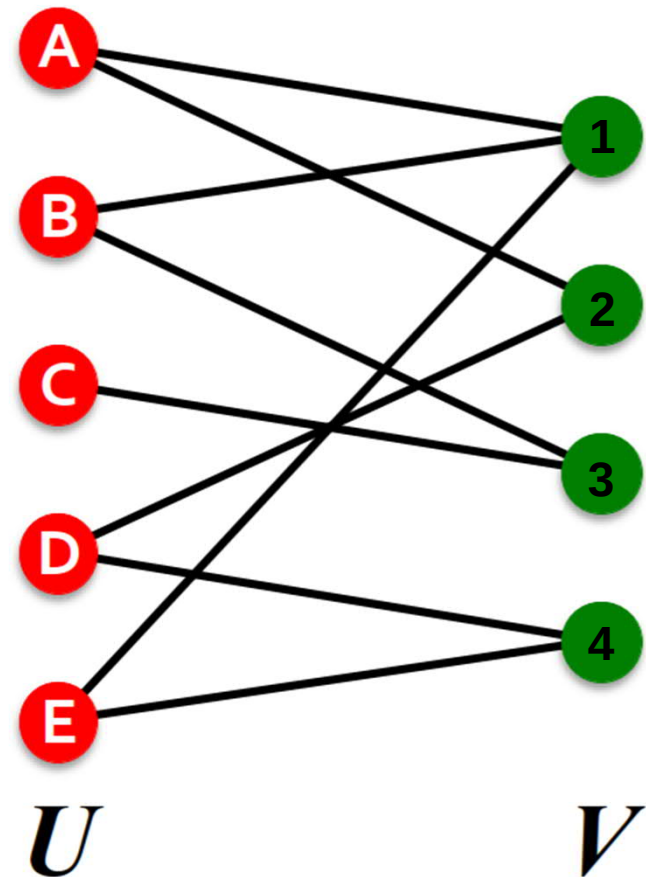
- ▶ Graphs can be directed or undirected



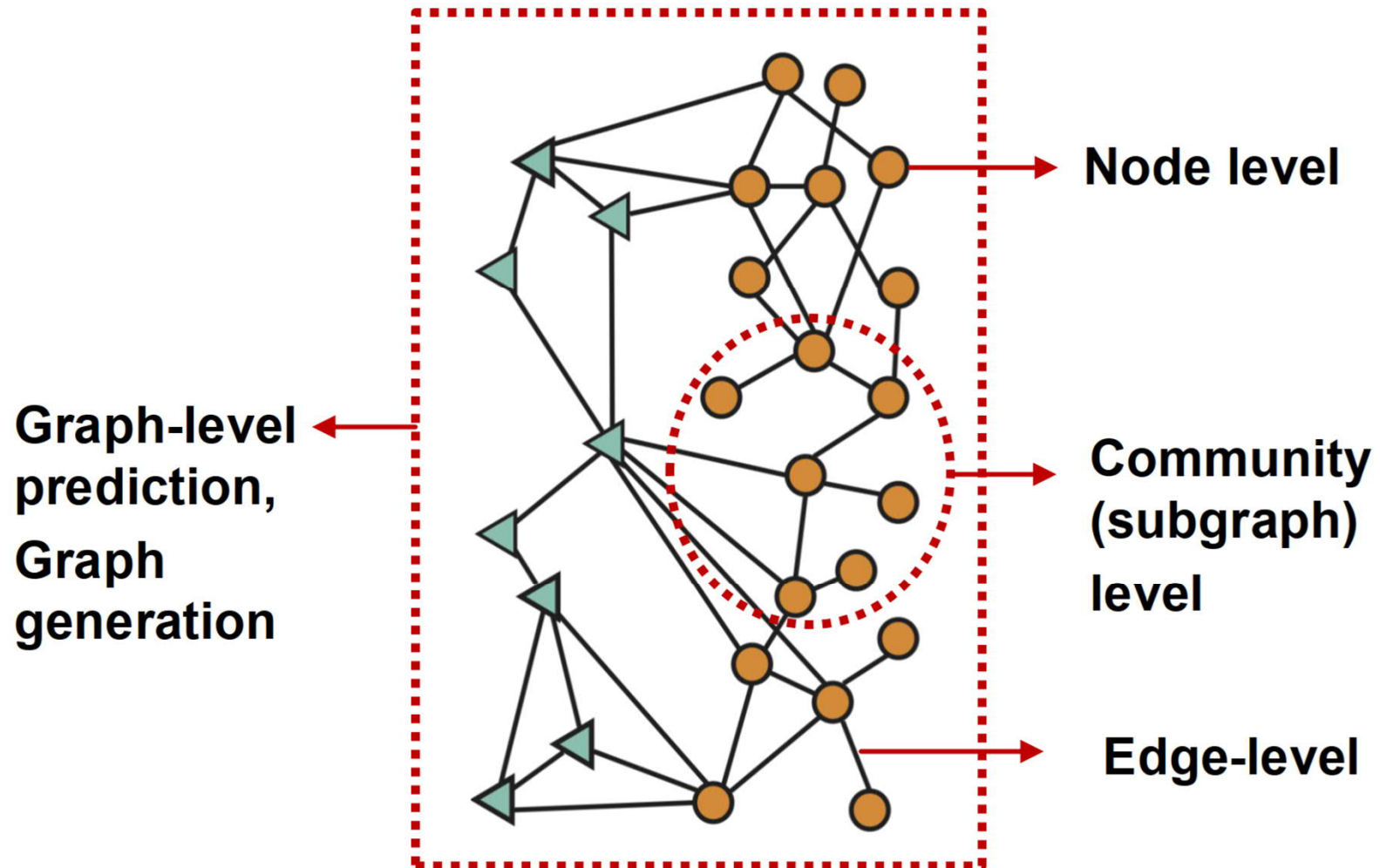
- ▶ Node degree:
 - ▶ Number of edges connecting the node
- ▶ In-degree of a node (for directed graphs):
 - ▶ Number of edges coming into the node
- ▶ Out-degree of a node (for directed graphs):
 - ▶ Number of edges going out of the node.

Bipartite Graph

- ▶ Nodes can be divided into two disjoint sets U and V such as every link connects a node in U to a node in V
- ▶ Examples:
 - ▶ Users-to-Items in a recommender system
 - ▶ Users-to-Movies in movie rating
 - ▶ Recipes-to-Ingredients

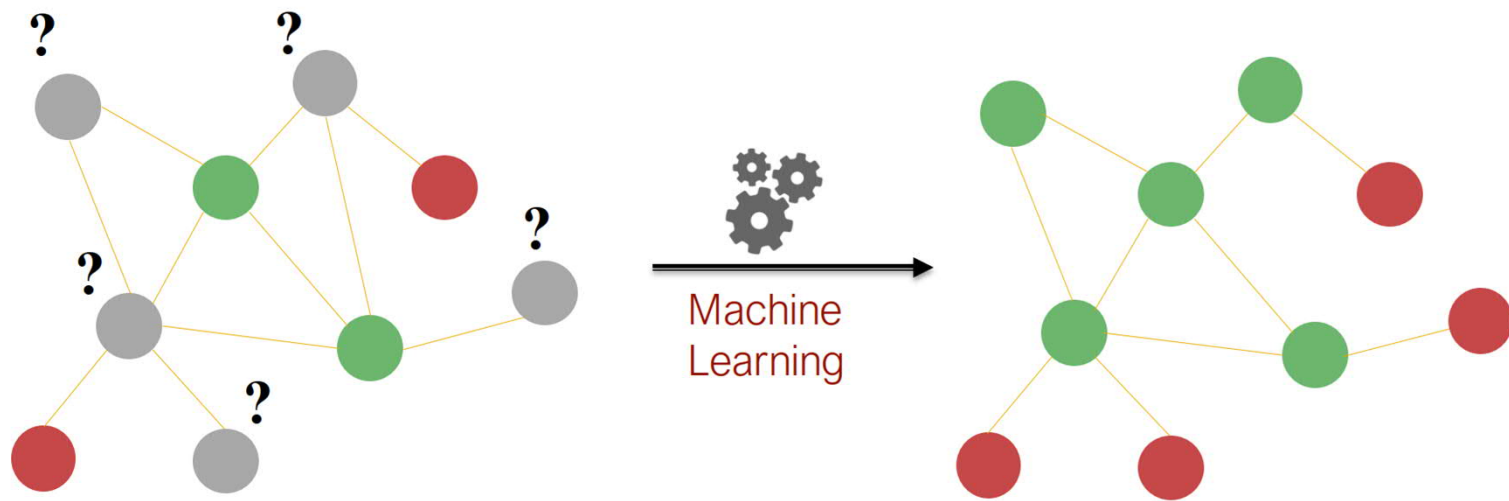


Different Types of Graph Tasks



Node-Level Tasks

▶ Node classification

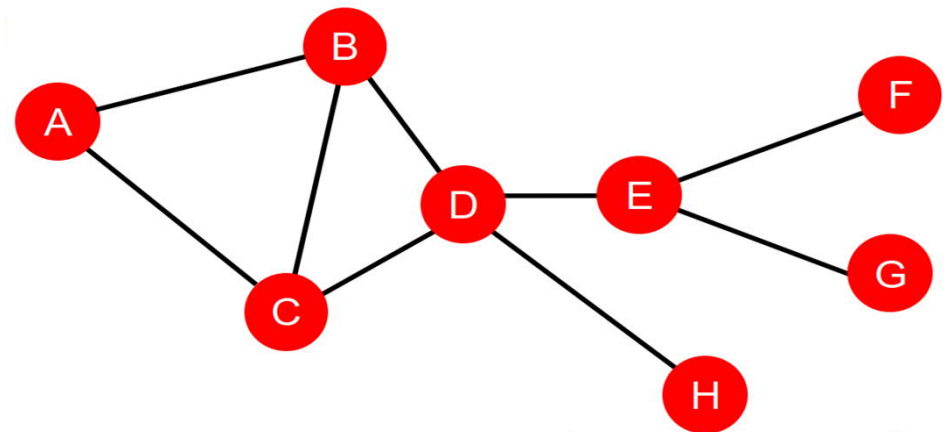


▶ Examples:

- ▶ Detecting fraudulent users in a financial network.
- ▶ Determining the political leaning of a social media user
- ▶ Identifying the function of a protein in a biological network

Node-Level Tasks

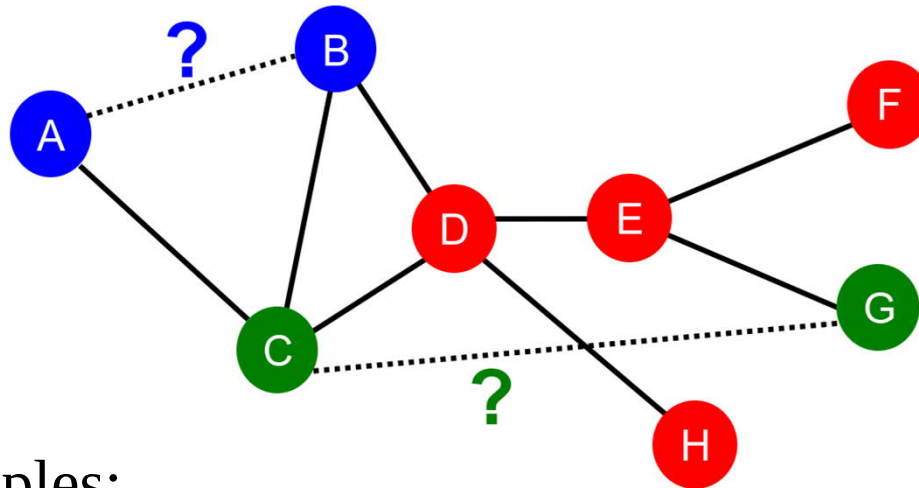
- ▶ Finding node importance by, e.g.,
 - ▶ Node degree
 - ▶ Number of shortest paths passing through the node
 - ▶ Average shortest path length to other nodes
 - ▶ Substructures around the node



Edge-Level Tasks

- ▶ Link prediction

- ▶ Predict new/missing/unknown links based on existing links

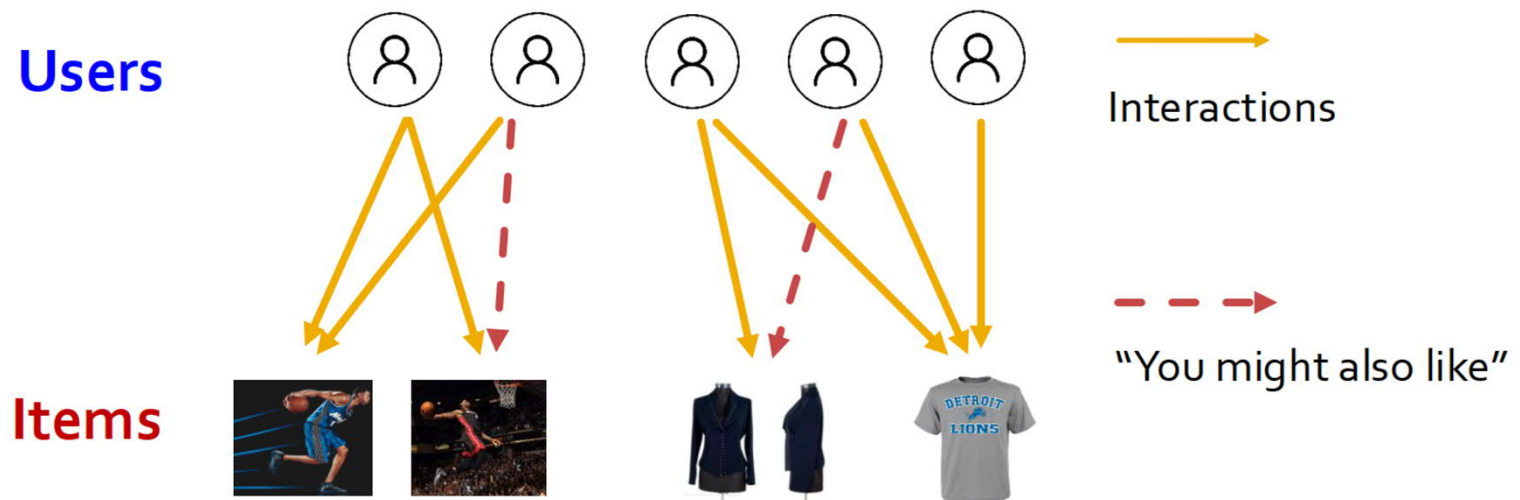


- ▶ Examples:

- ▶ Recommending friends in social networks
 - ▶ Predicting protein-protein interactions in biology
 - ▶ Recommending items to users in e-commerce systems
 - ▶ Identifying relationships in knowledge graphs

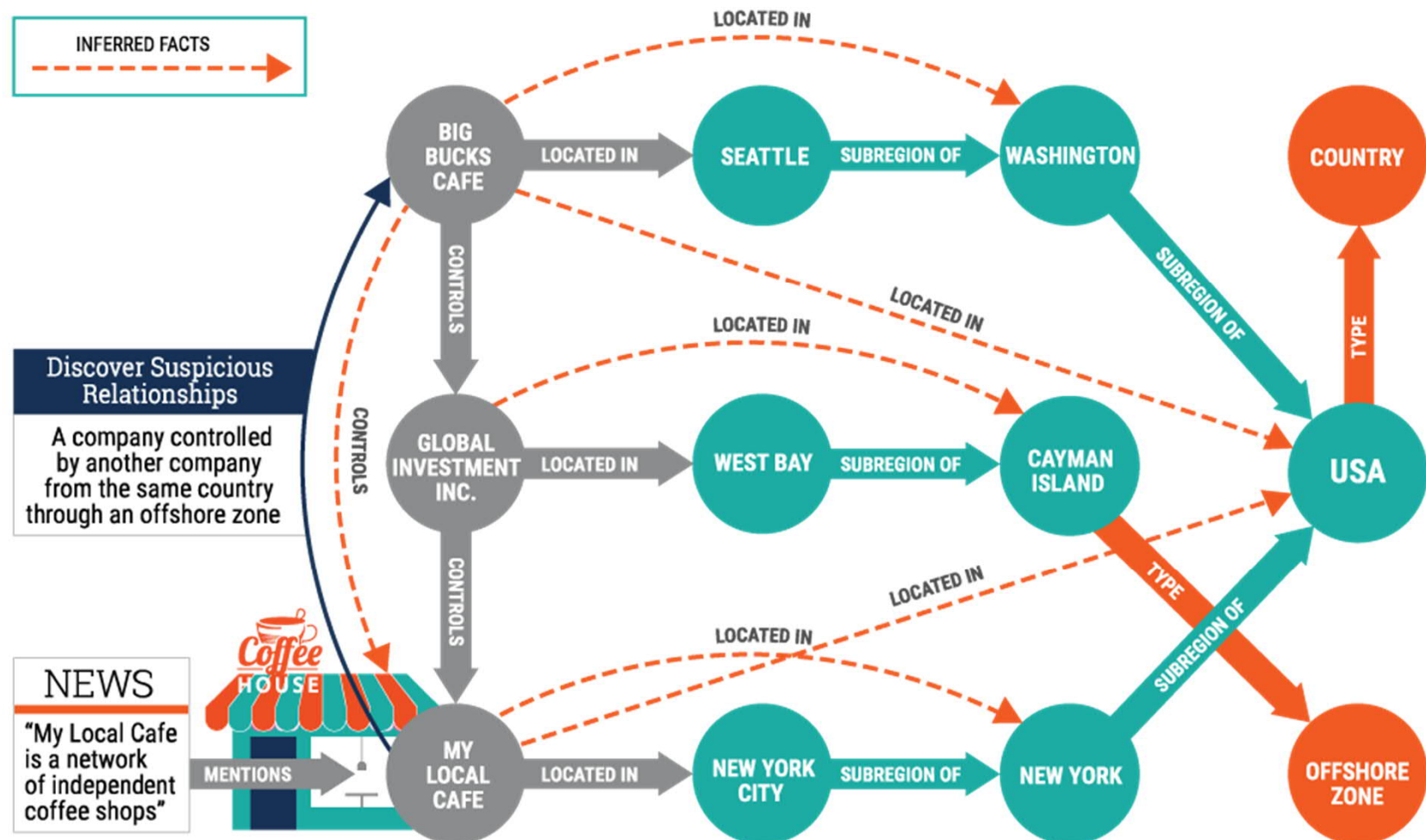
Link Prediction Examples

- ▶ Recommender systems
 - ▶ Nodes: users and items
 - ▶ Edges: a user bought an item
 - ▶ Goal: recommend items a user might like



Link Prediction Examples

- Identifying relationships in knowledge graphs



Graph-Level Tasks

- ▶ Goal: make a prediction for an entire graph or a subgraph

- ▶ Examples:

- ▶ Antibiotics prediction from promising molecules

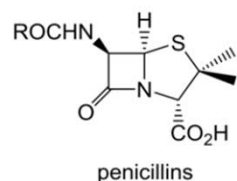
- ▶ Antibiotics are small molecular graphs

- ▶ Nodes: Atoms
 - ▶ Edges: Chemical bonds

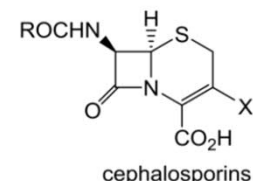
- ▶ Predicting whether a molecule is toxic

- ▶ Predicting the smell of a molecule

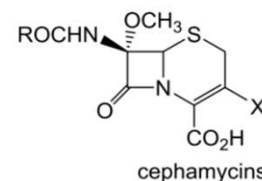
- ▶ ...



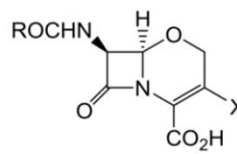
penicillins



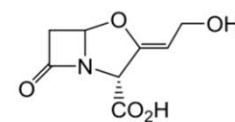
cephalosporins



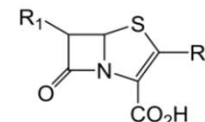
cephamycins



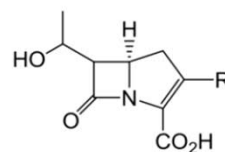
oxacephem



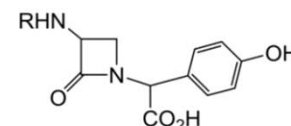
clavulanic acid
(an oxapenem)



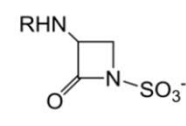
penems



carbapenems



nocardicin



monobactams

Konaklieva, Monika I. "Molecular targets of β -lactam-based antimicrobials: beyond the usual suspects." *Antibiotics* 3.2 (2014): 128-142.

Other Graph-Level Tasks

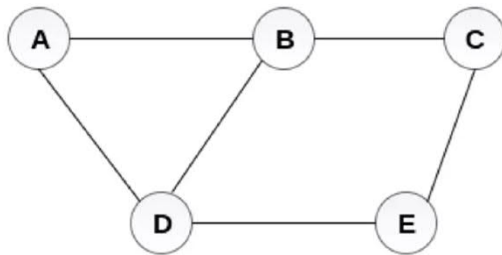
- ▶ Community Detection
 - ▶ Identify densely linked clusters of nodes
- ▶ Network similarity
 - ▶ How similar two (sub)networks are

Graph Representation Methods

- ▶ Explicit representation
 - ▶ Encode the exact connectivity between nodes
 - ▶ Types:
 - ▶ Adjacency matrix
 - ▶ Adjacency list
 - ▶ Edge list
- ▶ Implicit representation
 - ▶ Represent nodes, edges, or entire graphs as low-dimensional numerical vectors, called *graph embeddings*
 - ▶ These are learned representations
 - ▶ Preserve structural and semantic similarity

Adjacency matrix

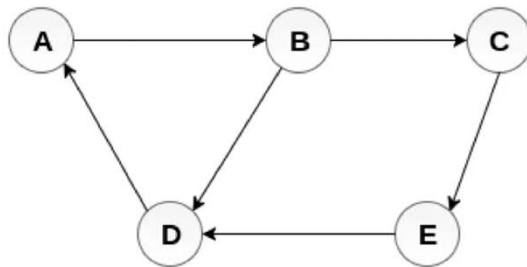
- ▶ $N \times N$ matrix (N is the number of nodes)



Undirected Graph

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	1	0
C	0	1	0	0	1
D	1	1	0	0	1
E	0	0	1	1	0

Adjacency Matrix



Directed Graph

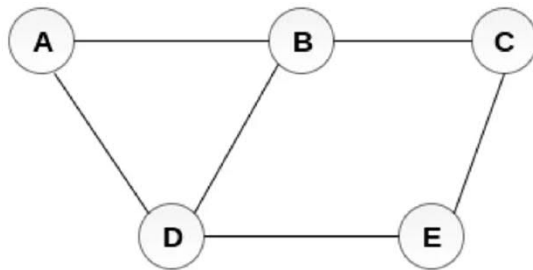
	A	B	C	D	E
A	0	1	0	0	0
B	0	0	1	1	0
C	0	0	0	0	1
D	1	0	0	0	0
E	0	0	0	1	0

Adjacency Matrix

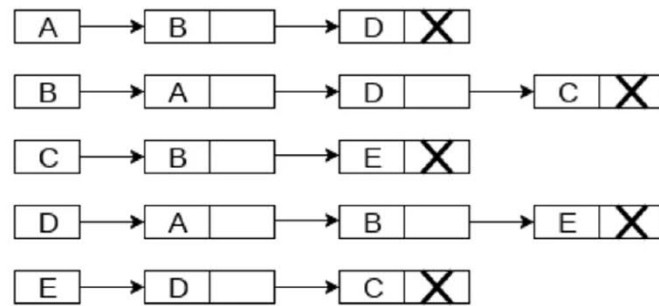
If there are weights on edges, weights can be used instead of binary numbers.

Adjacency list

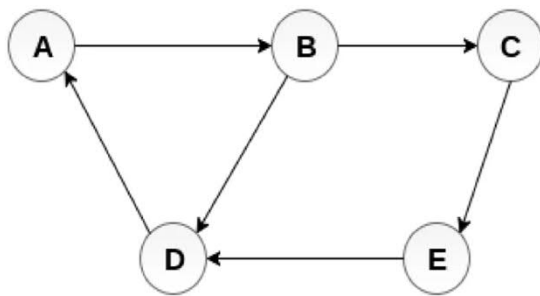
- ▶ Space efficient for sparse matrix



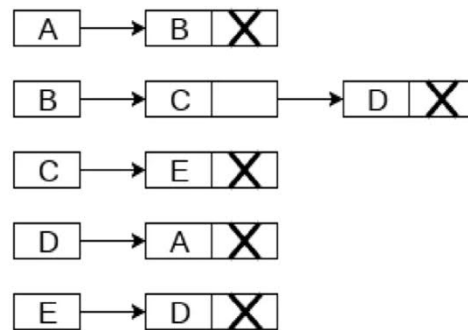
Undirected Graph



Adjacency List



Directed Graph



Adjacency List

Graph Embeddings

- ▶ Represent nodes/edges/graphs using low-dimensional *numerical vectors*
- ▶ Capture *structural* patterns and *semantic* relationships,
- ▶ Enable machine-learning models to operate on graphs efficiently while preserving important topological information.
- ▶ Embeddings are *learned* representations

Types of Graph Embedding Learning Methods

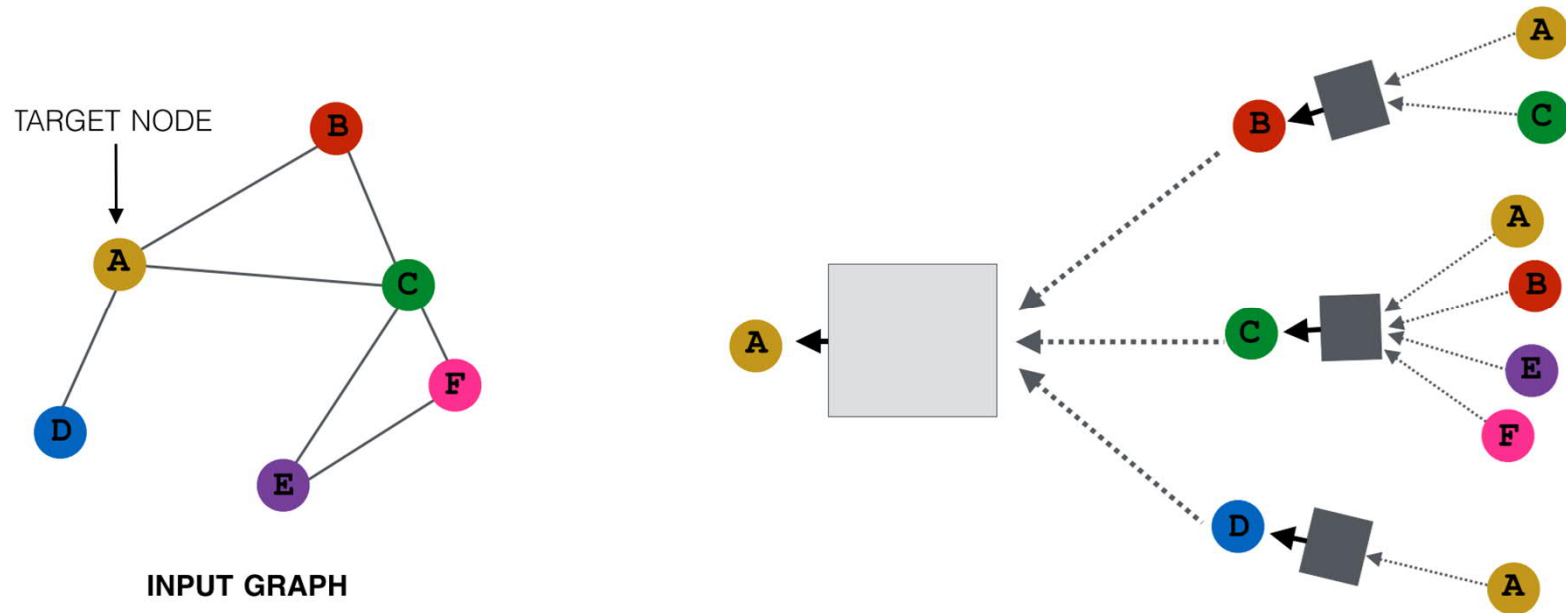
- ▶ Traditional
 - ▶ Node2Vec
 - ▶ DeepWalk
 - ▶ ...
- ▶ Graph neural network (GNN) based
 - ▶ Graph Convolutional Network (GCN)
 - ▶ GraphSAGE
 - ▶ ...

Graph Neural Networks

- ▶ Deep learning model that generates a node embedding by aggregating the node's neighbors' embeddings.
- ▶ Intuition:
 - ▶ A node's state is influenced by its interactions with its neighbors in the graph.

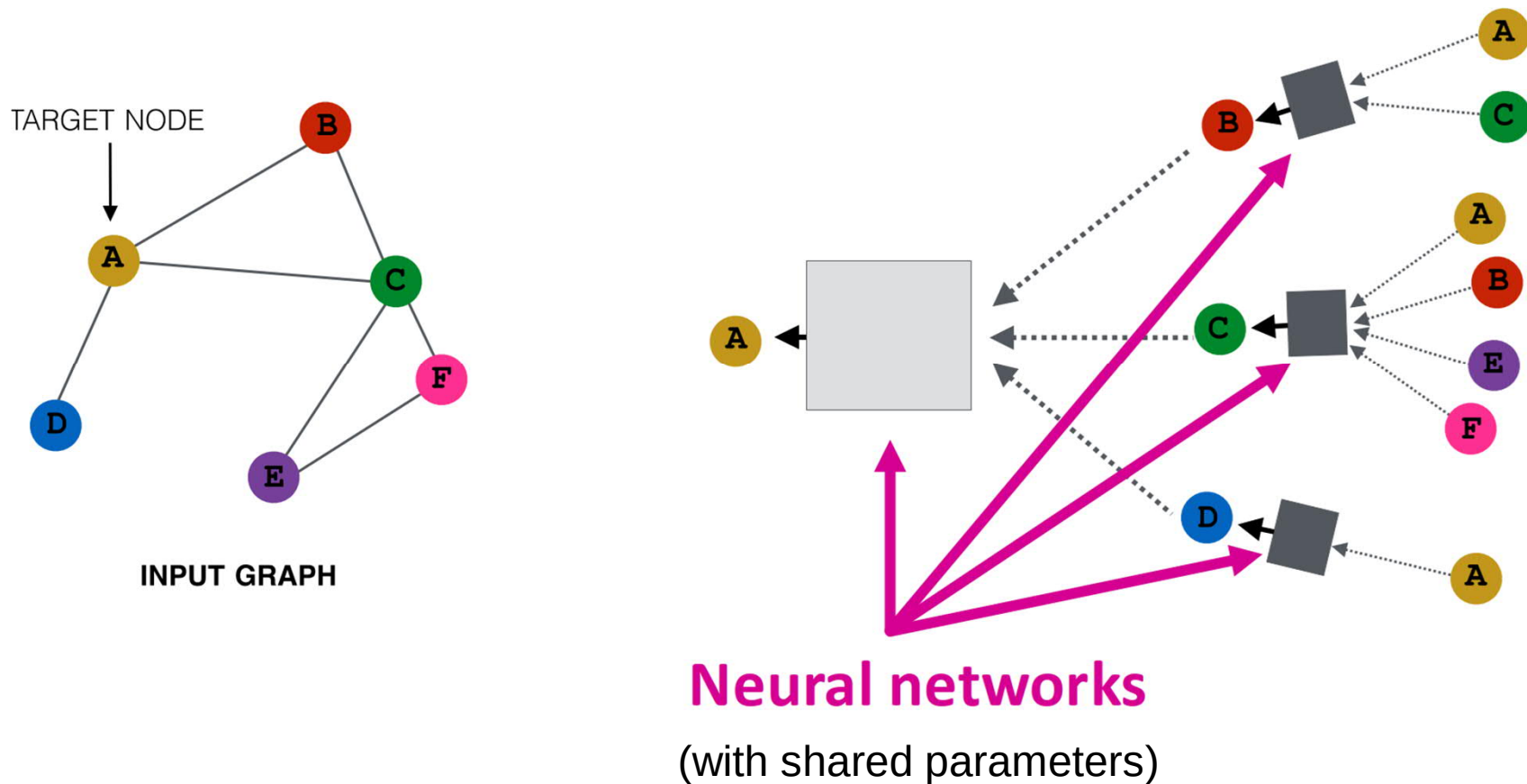
Graph Neural Networks

- ▶ Key idea:
 - ▶ Generate node embeddings by aggregating neighbors



Graph Neural Networks

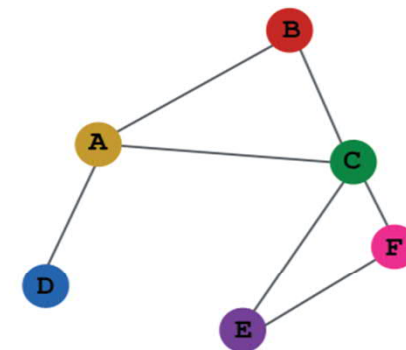
- ▶ Nodes aggregate information from their neighbors using neural networks



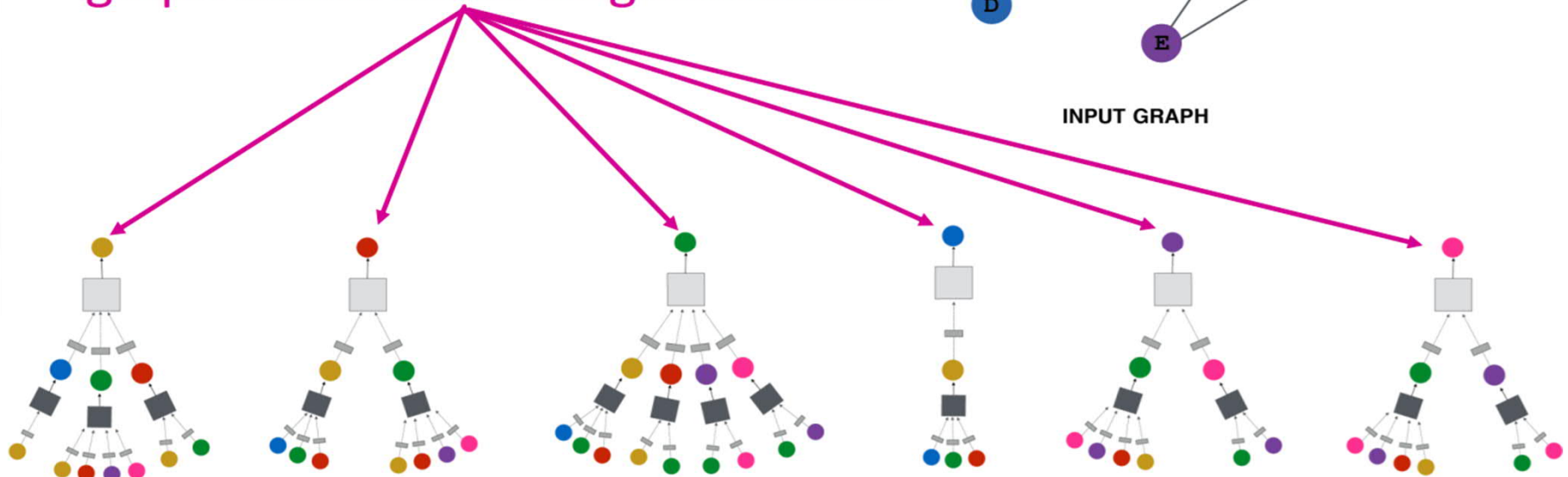
Graph Neural Networks

- ▶ Network neighborhood of a node defines its computation graph

Every node defines a computation graph based on its neighborhood!

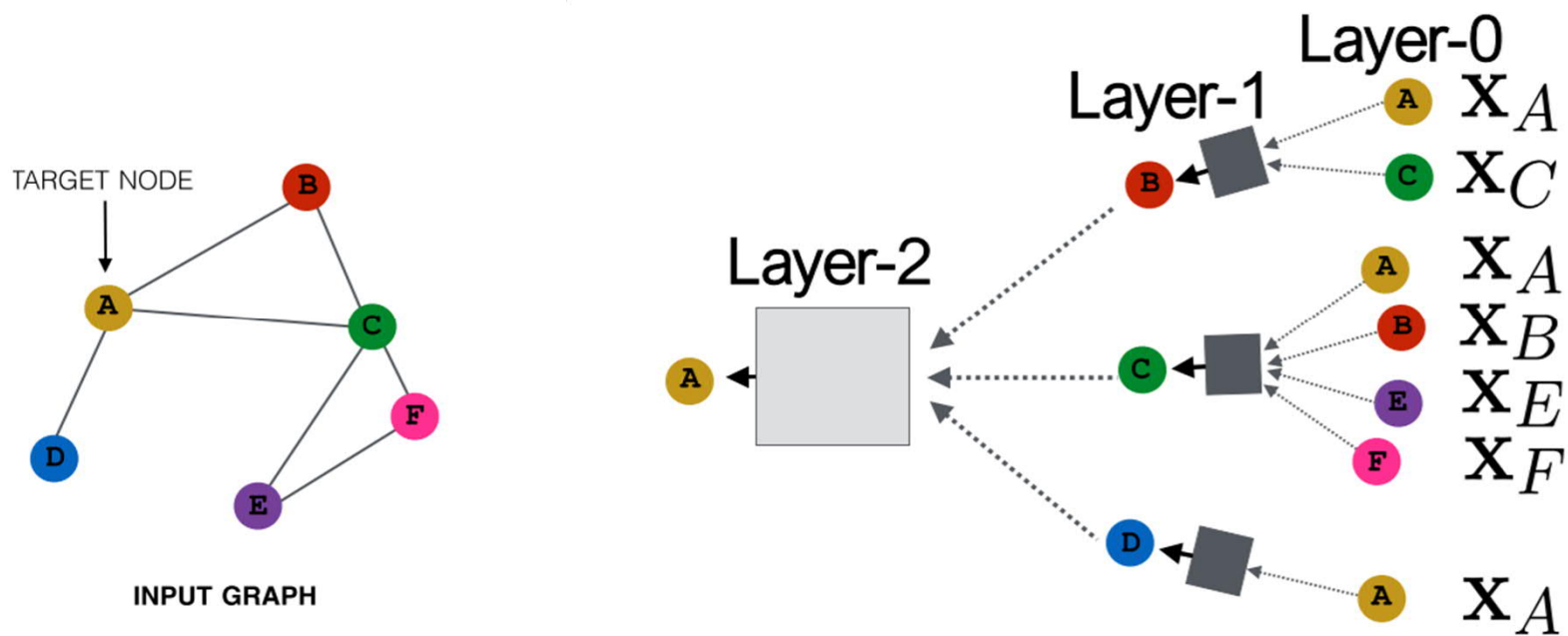


INPUT GRAPH



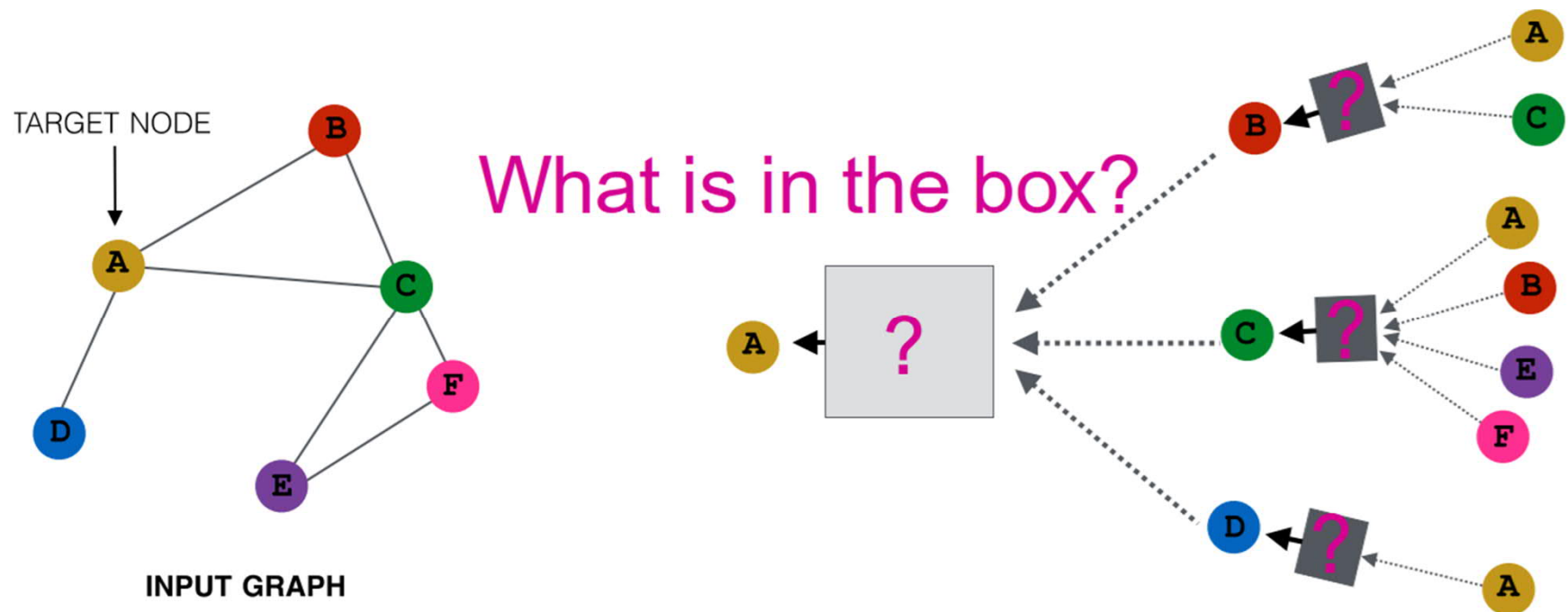
Graph Neural Networks

- ▶ Graph can be of arbitrary depth:
 - ▶ Nodes have embeddings at each layer
 - ▶ Layer-0 embedding of node v is its input feature x_v
 - ▶ Layer- k embedding gets information from nodes that are k hops ways



Neighborhood Aggregation

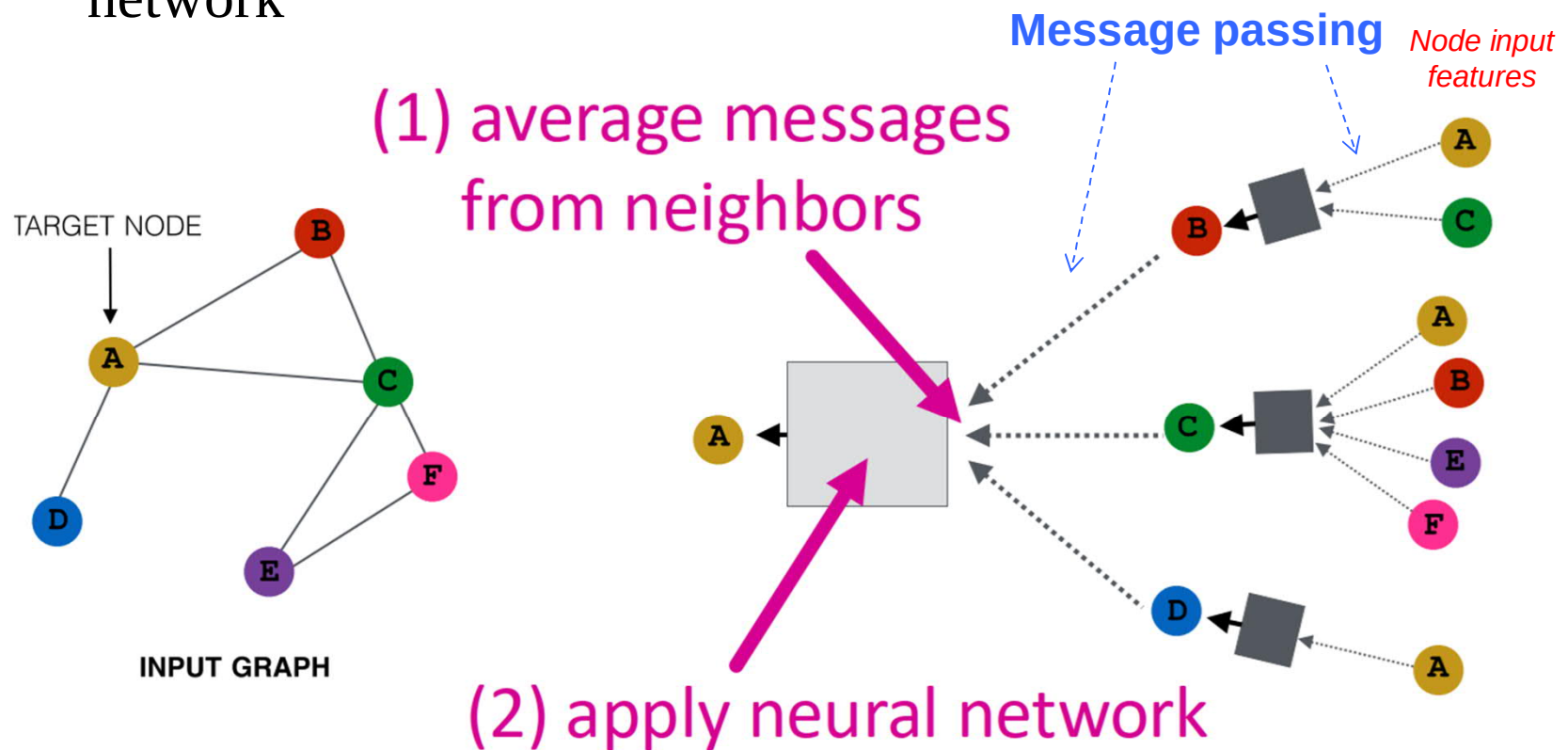
- ▶ Different approaches may aggregate information across the layers differently



How to Compute the Embedding of a Node

► Basic approach:

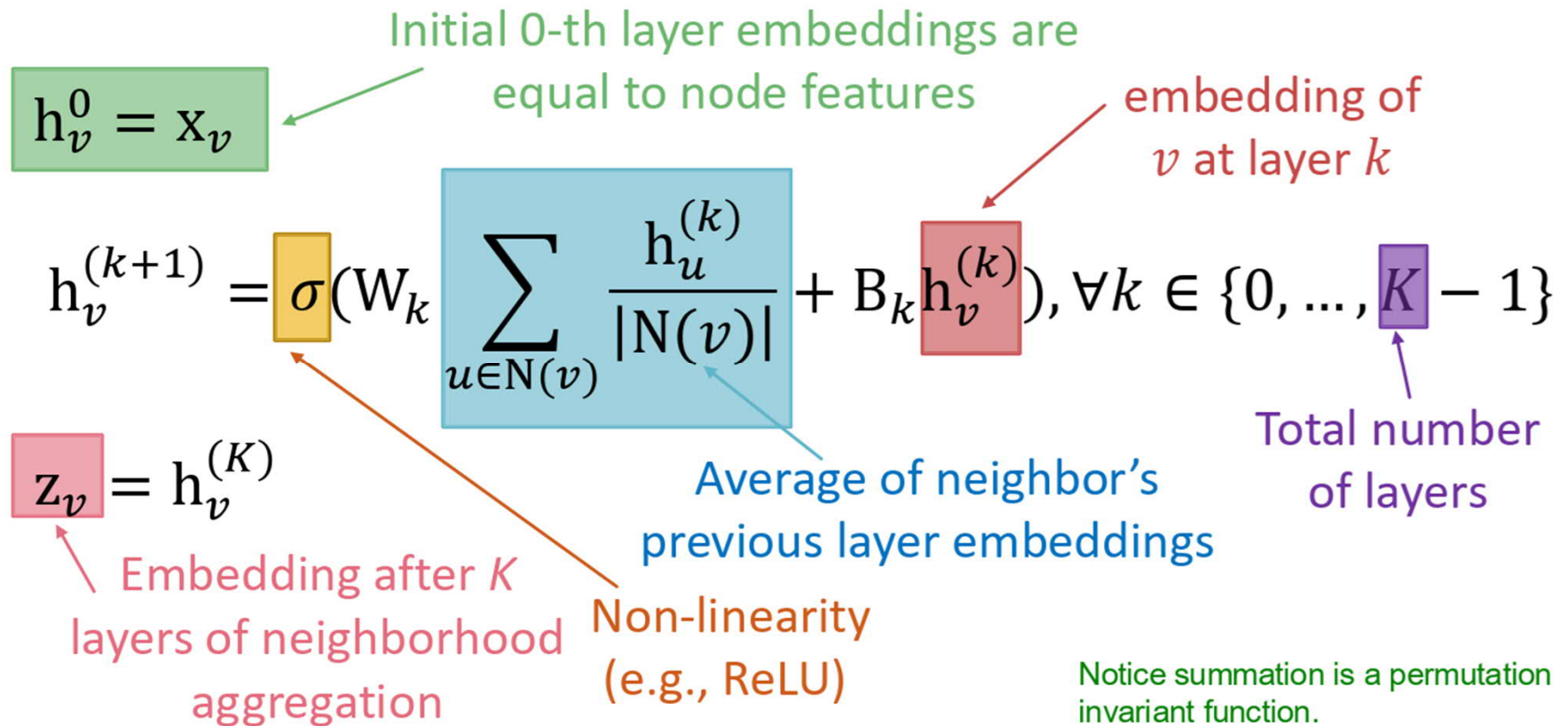
- Average information from neighbors and apply a neural network



How to Compute the Embedding of a Node

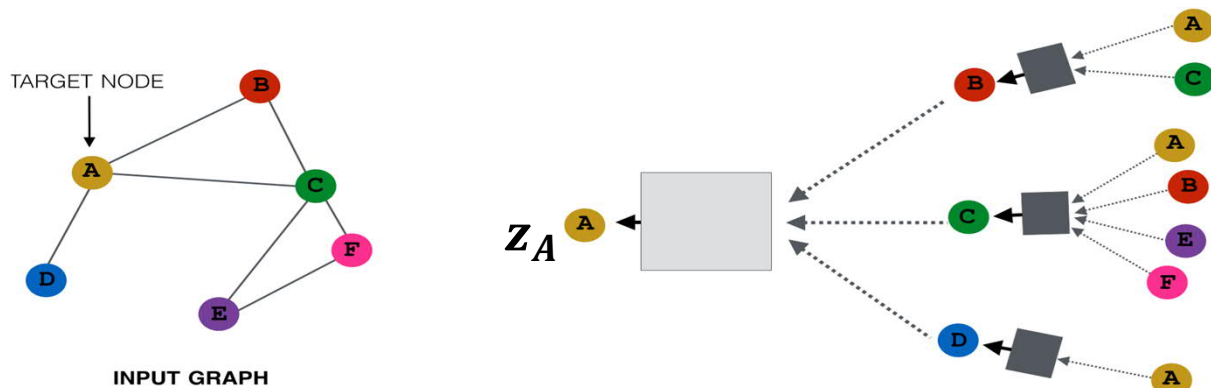
► Math:

- Average information from neighbors and apply a neural network



How to Train the Model

- ▶ What to train to generate node embeddings



Trainable weight matrices
(i.e., what we learn)

$$h_v^{(0)} = x_v$$

$$h_v^{(k+1)} = \sigma\left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)}\right), \forall k \in \{0..K-1\}$$

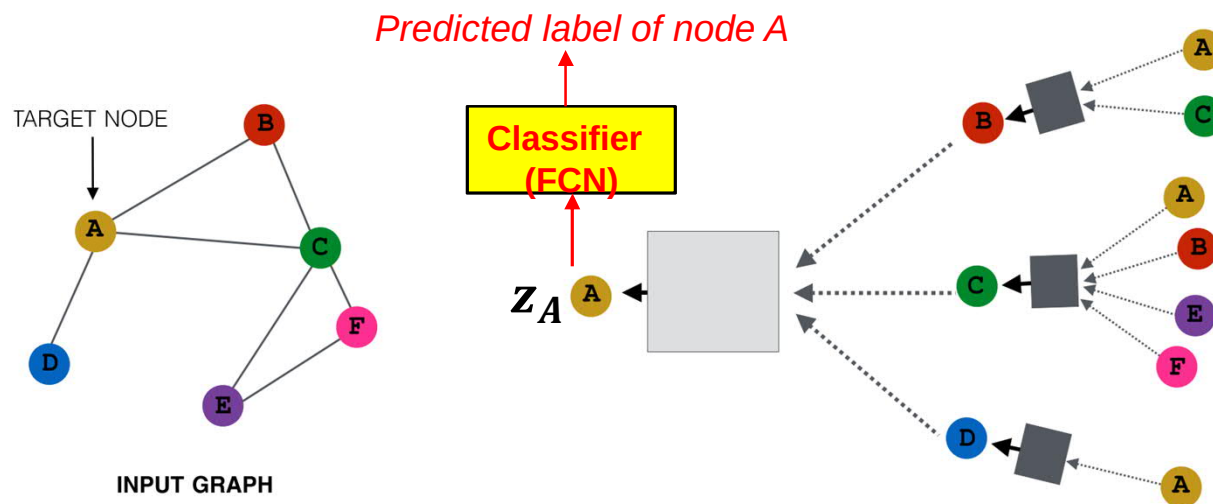
Final node embedding

$$z_v = h_v^{(K)}$$

W_k is the matrix for converting the aggregated previous-layer embeddings from neighbors
 B_k is the matrix for transforming its own vector from the previous layer

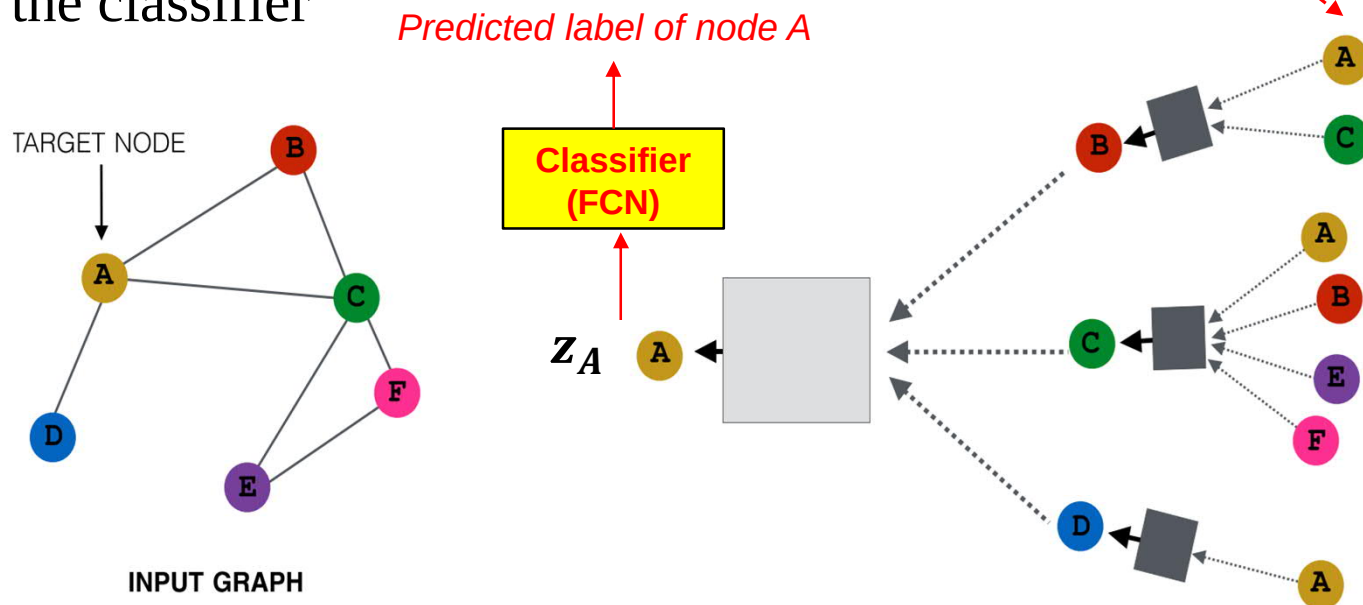
How to Train the Model

- ▶ One way to train the model is to use the node classification task
 - ▶ If some nodes have a class label (e.g., if the node is a fraudulent user, or the functional role of a protein, etc.)
 - ▶ These labels serve as the ground truth
- ▶ Add a **classifier** (one or two fully-connected layers) on top of the final embedding layer



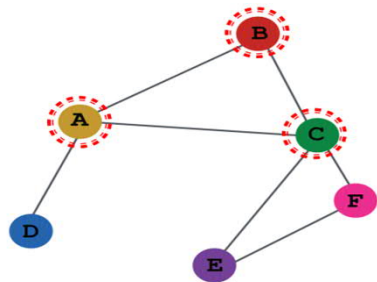
How to Train the Model

- ▶ Given a node with class label,
 - ▶ Use the input vectors of its K-hop neighbors as input
 - ▶ Go through the GCN encoder (i.e., K layers transformation) with the current weights to get the embedding $\mathbf{z}_A$
 - ▶ Feed $\mathbf{z}_A$ into the classifier to obtain the predicted label for A
 - ▶ Compute the loss and backpropagate to update all weights in the GNN and the classifier

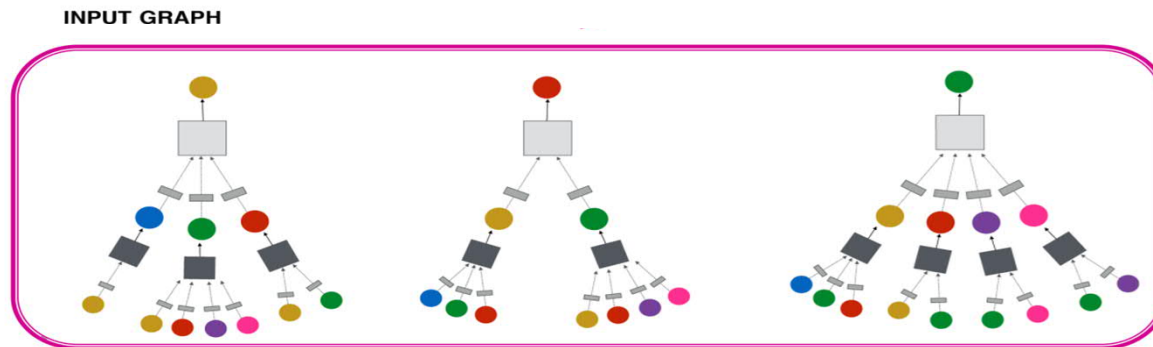


How to Train the Model

- ▶ Training goes through all (or a subset of) the nodes that have class labels
 - ▶ Each node is like a training example
 - ▶ Each node has its own *computation graph*
- ▶ Can go with multiple epochs (i.e., go through all the labeled nodes multiple times).



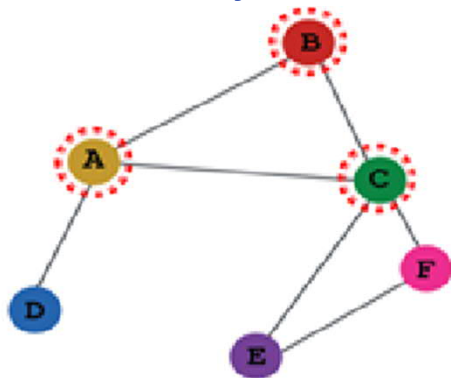
Computation graphs for nodes A, B and C:



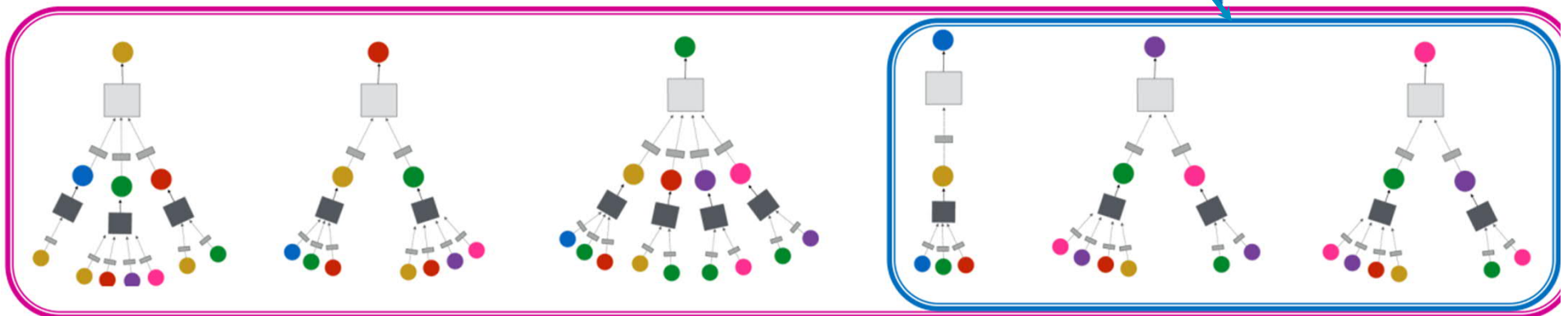
Inductive Capability of GNN

- ▶ After training, we
 - ▶ not only obtain the embeddings of the nodes with class labels
 - ▶ but also embeddings of the nodes without class labels

Assume that only A, B and C are labeled:

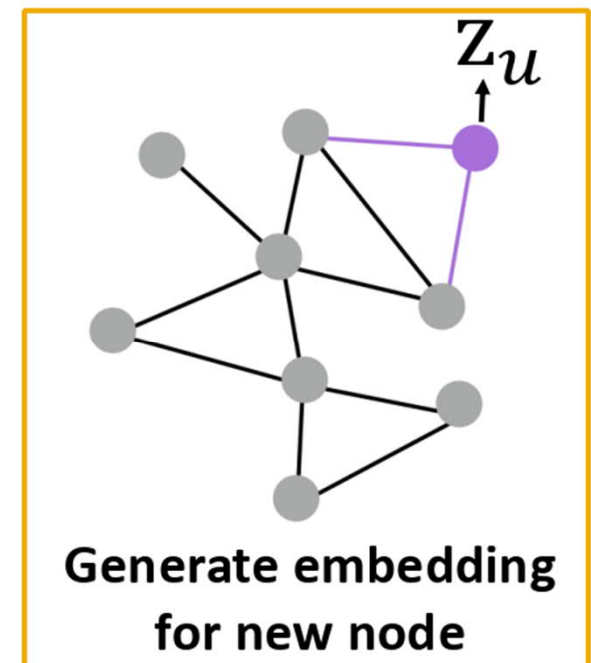
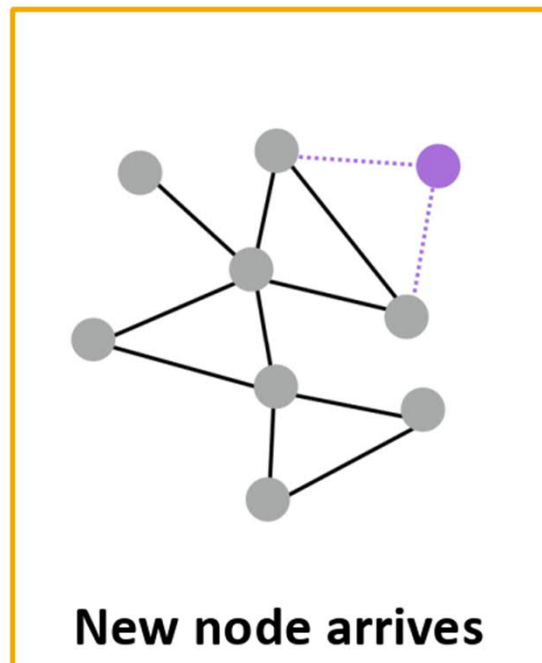
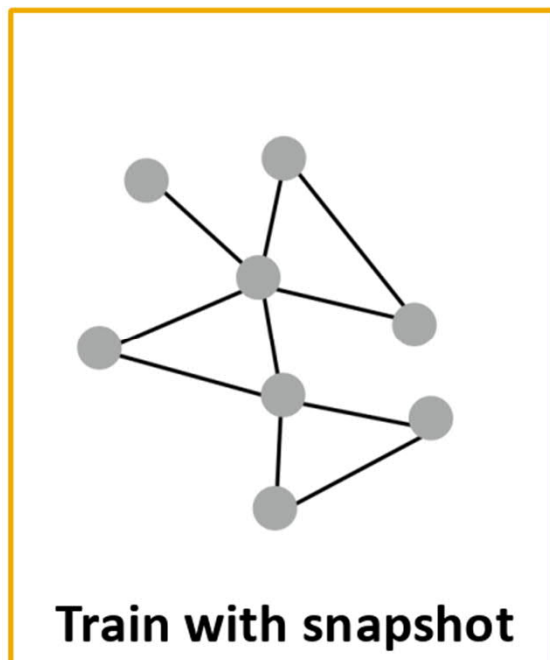


Embeddings of nodes D, E and F are also obtained through the training:



Inductive Capability of GNN: New Nodes

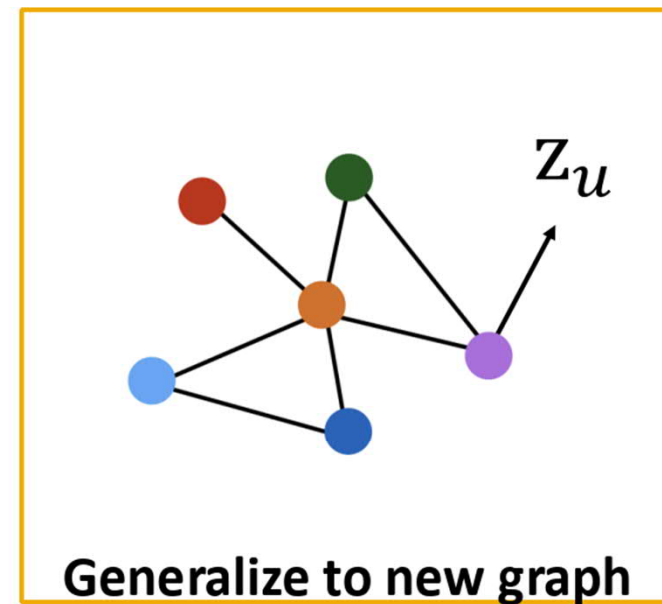
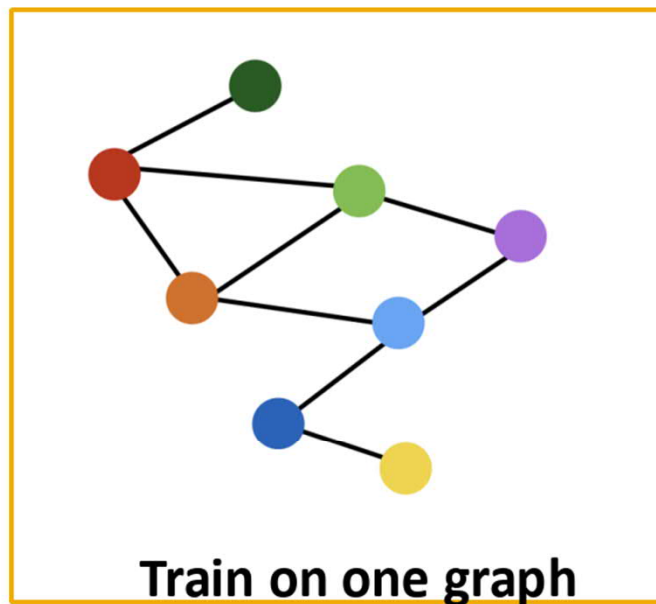
- ▶ Can generate embeddings on **new/unseen nodes**



- Many application settings constantly encounter previously unseen nodes:
 - E.g., Reddit, YouTube, Google Scholar
- Need to generate new embeddings “on the fly”

Inductive Capability of GNN: New Graphs

- ▶ Can generate embeddings on **new graphs**



Inductive node embedding → Generalize to entirely unseen graphs

E.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Usefulness of Node Embeddings

- ▶ With the trained node embeddings, we can
 - ▶ Compute edge embeddings (e.g., by averaging the nodes embeddings)
 - ▶ Sub-graph embeddings
- ▶ Perform graph tasks:
 - ▶ Node classification
 - ▶ Node clustering (Community detection)
 - ▶ Link prediction
 - ▶ Graph similarity computation
 - ▶ ...

Advantages of GNN for Graph Embeddings over Traditional Graph Embedding Methods

- ▶ Can consider node features
 - ▶ Previous methods (e.g., Node2Vec, based on random walk) cannot incorporate node features. They only consider the graph structure
- ▶ Can incorporate edge weights.
- ▶ Inductive:
 - ▶ Can generate embeddings for new nodes and node graphs

Summary

- ▶ Graph fundamentals
- ▶ Graph mining tasks
- ▶ Graph representation methods
- ▶ Graph embeddings
- ▶ GNN basics
- ▶ How to train a GNN
- ▶ Advantages of GNNs

Acknowledgements

Some slides are adapted from slides by Jure Leskovec of Stanford University.